

PROJECT: LINUX SYSTEM MONITORING & AUTOMATED ALERTING TOOL

Table of Contents

1. Project Overview
2. Objectives
3. Environment and Technologies
4. Project Architecture
5. Directory Structure
6. Implementation
7. Monitoring Features
8. Threshold-Based Alerting
9. Logging and Report Generation
10. systemd Automation
11. Troubleshooting and Challenges
12. Testing and Validation
13. Screenshots to Add
14. Security Considerations
15. Project Outcome and Skills Demonstrated
16. Future Enhancements
17. Conclusion

1. Project Overview

The Linux System Monitoring & Automated Alerting Tool is a Bash-based monitoring utility developed on Red Hat Enterprise Linux 9.8. The project collects important system health information, checks configurable resource thresholds, monitors selected services and network connectivity, records monitoring output, and automates execution using systemd.

The project is designed to demonstrate practical Linux system administration concepts rather than relying only on existing monitoring applications. It combines Bash scripting with standard Linux commands, the /proc filesystem, systemd, logging, permissions, and SELinux troubleshooting.

2. Objectives

- Monitor basic Linux system information such as hostname, operating system, kernel and uptime.
- Monitor CPU utilization, load average and memory utilization.
- Monitor filesystem utilization and identify filesystems exceeding a configured threshold.
- Display top processes by CPU utilization.
- Monitor network interfaces, default route and Internet connectivity.
- Check the operational state of critical services such as sshd and firewalld.
- Generate WARNING and CRITICAL alerts when monitored conditions exceed defined limits or fail.
- Maintain a historical monitoring log and generate a current system report.
- Automate monitoring through a systemd service and timer.
- Troubleshoot real Linux security and execution issues, including an SELinux denial.

3. Environment and Technologies

Component	Details
Operating System	Red Hat Enterprise Linux 9.8 (Plow)
Virtualization	VMware virtual machine
Shell	Bash
Monitoring utilities	top, free, awk, df, ps, ip, ping, systemctl
Configuration	Bash configuration file (monitor.conf)
Automation	systemd service and systemd timer
Security	SELinux
Project path	/linux-system-monitor/

The attached reference document uses a structured Linux-project format covering project overview, objectives, environment, implementation steps, purpose, commands, explanations, troubleshooting and conclusion. This documentation follows the same practical, step-by-step style while adapting it to the monitoring project.

4. Project Architecture

```
systemd timer
|
v
linux-monitor.service
|
v
scripts/monitor.sh
```

```
|
+--> System information
+--> CPU / Load
+--> Memory
+--> Disk
+--> Top processes
+--> Network
+--> sshd / firewalld
|
+--> Threshold alerts
|
+--> logs/monitor.log
+--> reports/system-reports.txt
```

5. Directory Structure

```
/linux-system-monitor/
├── config/
│   └── monitor.conf
├── logs/
│   └── monitor.log
├── reports/
│   └── system-reports.txt
├── scripts/
│   └── monitor.sh
└── systemd/
    ├── linux-monitor.service
    └── linux-monitor.timer
```

6. Implementation

Step 1 - Create the Project Structure

Purpose: Separate configuration, scripts, logs, reports and systemd unit files.

```
# cd /
# sudo mkdir -p /linux-system-monitor/{config,logs,reports,scripts,systemd}
# sudo chown -R kunjan:kunjan /linux-system-monitor
# cd /linux-system-monitor
```

Explanation: The structure keeps the project organized and makes the monitoring tool easier to maintain and document.

Step 2 - Create the Monitoring Script

Purpose: Build the main Bash script responsible for collecting system health information.

```
# vim scripts/monitor.sh
# chmod +x scripts/monitor.sh
```

Explanation: The script acts as the central monitoring engine.

Step 3 - Collect System Information

Purpose: Display hostname, OS version, kernel and uptime.

```
# hostname
# grep '^PRETTY_NAME=' /etc/os-release
# uname -r
# uptime -p
```

Explanation: These commands provide a quick baseline of the monitored host.

Step 4 - Monitor CPU, Load and Memory

Purpose: Measure current CPU utilization, load average and memory utilization.

```
# top -bn1
# awk '{print $1}' /proc/loadavg
# free
```

Explanation: top provides CPU statistics, /proc/loadavg provides load information, and free provides memory statistics. The script converts the relevant values into readable percentages.

Step 5 - Monitor Disk and Processes

Purpose: Identify filesystem utilization and display processes consuming the most CPU.

```
# df -h --output=target,pcent
# ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head -6
```

Explanation: The script evaluates filesystem percentages and displays the top CPU-consuming processes.

Step 6 - Monitor Network Connectivity

Purpose: Display interfaces, default route and test connectivity.

```
# ip -br addr
# ip route | grep default
# ping -c 2 -W 2 8.8.8.8
```

Explanation: The network section provides interface status, routing information and a basic connectivity test.

Step 7 - Monitor Critical Services

Purpose: Verify that important services are running.

```
# systemctl is-active --quiet sshd
# systemctl is-active --quiet firewalld
```

Explanation: The script reports RUNNING or DOWN and produces a CRITICAL alert when a monitored service is not active.

Step 8 - Configure Monitoring Thresholds

Purpose: Keep alert limits separate from the monitoring logic.

```
# config/monitor.conf
CPU_THRESHOLD=80
MEMORY_THRESHOLD=80
DISK_THRESHOLD=80
```

Explanation: Thresholds can be changed without rewriting the monitoring logic.

Step 9 - Add Alerts

Purpose: Convert collected measurements into actionable status messages.

```

[OK] CPU usage is normal
[WARNING] CPU usage is above 80%
[OK] Memory usage is normal
[WARNING] Memory usage is above 80%
[CRITICAL] firewalld service is not running
[CRITICAL] Network connectivity failed

```

Explanation: Alerts make the tool useful for troubleshooting instead of only displaying raw metrics.

Step 10 - Configure Logging and Reporting

Purpose: Preserve monitoring history and create a current report.

```

# LOG_FILE="/linux-system-monitor/logs/monitor.log"
# REPORT_FILE="/linux-system-monitor/reports/system-reports.txt"
# exec > >(tee -a "$LOG_FILE" | tee "$REPORT_FILE") 2>&1

```

Explanation: The historical log is appended on each run, while the report file contains the latest run.

7. Monitoring Features

Feature	Implementation
System baseline	Hostname, RHEL version, kernel version and uptime.
CPU	Current CPU utilization with configurable threshold validation.
Load average	Reads the one-minute load value from /proc/loadavg.
Memory	Calculates memory utilization and validates it against a threshold.
Disk	Checks mounted filesystems and warns when utilization exceeds the configured limit.
Processes	Lists the top processes sorted by CPU usage.
Network	Displays interfaces, default route and connectivity status.
Services	Checks sshd and firewalld.
Alerts	Provides OK, WARNING and CRITICAL messages.
Logging	Maintains a historical monitoring log.
Reporting	Creates a current system report.
Automation	Runs through systemd on a schedule.

8. Threshold-Based Alerting

The tool uses a separate monitor.conf file so administrators can adjust thresholds according to the environment. The implemented baseline thresholds are 80% for CPU, memory and disk usage.

Example CPU alert logic:

```

if (( $(echo "$CPU_USAGE > $CPU_THRESHOLD" | bc -l) ))
then
    echo "[WARNING] CPU usage is above ${CPU_THRESHOLD}%"
else
    echo "[OK] CPU usage is normal"
fi

```

Disk and service conditions are handled with integer comparisons and systemctl state checks. This creates simple, readable operational alerts suitable for a small Linux administration environment.

9. Logging and Report Generation

The monitoring output is written to two destinations. monitor.log acts as the historical record, while system-reports.txt represents the latest report. The use of tee allows the same output to remain visible in the terminal while being stored on disk.

```
#btail -20 logs/monitor.log
# cat reports/system-reports.txt
# ls -lh logs/monitor.log reports/system-reports.txt
```

10. systemd Automation

Service unit:

```
[Unit]
Description=Linux System Monitoring Tool
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/linux-system-monitor/scripts/monitor.sh
User=kunjan
```

Timer unit:

```
[Unit]
Description=Run Linux System Monitor Every 5 Minutes

[Timer]
OnBootSec=1min
OnUnitActiveSec=5min
Unit=linux-monitor.service
Persistent=true

[Install]
WantedBy=timers.target
```

Installation and activation:

```
# sudo cp systemd/linux-monitor.service /etc/systemd/system/
# sudo cp systemd/linux-monitor.timer /etc/systemd/system/
# sudo systemctl daemon-reload
# sudo systemctl enable --now linux-monitor.timer
# systemctl status linux-monitor.timer
# systemctl list-timers --all | grep linux-monitor
```

The timer triggers the oneshot service. The service executes monitor.sh once, then exits. This design is appropriate because the timer controls the monitoring frequency.

11. Troubleshooting and Challenges

Challenge 1 - Log file permission denied

Symptom: Running the script produced a tee permission error when writing to monitor.log. The log file had been created with sudo and was therefore owned by root.

```
tee: /linux-system-monitor/logs/monitor.log: Permission denied
```

Solution:

```
# sudo chown -R kunjan:kunjan /linux-system-monitor
# sudo chmod 755 /linux-system-monitor/logs
# sudo chmod 644 /linux-system-monitor/logs/monitor.log
```

Verification showed monitor.log owned by kunjan and the file size increased after running the script.

Challenge 2 - Report file remained empty

Symptom: The report file existed but had zero bytes. The logging and report paths were separated so the report needed to be explicitly populated.

Solution: Route the monitoring output through tee for both the historical log and the current report:

```
exec > >(tee -a "$LOG_FILE" | tee "$REPORT_FILE") 2>&1
```

This preserves historical output in monitor.log and refreshes system-reports.txt with the latest run.

Challenge 3 - systemd service failed with 203/EXEC

Symptom: systemd could not execute monitor.sh and returned status 203/EXEC with Permission denied.

```
Failed at step EXEC spawning /linux-system-monitor/scripts/monitor.sh: Permission denied
status=203/EXEC
```

Investigation:

- Verified the script had executable permissions: -rwxr-xr-x.
- Verified the project and scripts directories were accessible to kunjan.
- Confirmed the root filesystem was not mounted with noexec.
- Confirmed the script could be executed as the kunjan user.
- Checked SELinux enforcement and audit logs.

Root cause:

SELinux was enforcing a policy that denied systemd execution of the script because the script had the default_t SELinux file type.

```
# getenforce
Enforcing

# ls -Z /linux-system-monitor/scripts/monitor.sh
...:object_r:default_t:s0

# sudo ausearch -m AVC -ts recent | tail -20
avc: denied { execute } ... tcontext=...:default_t:s0
```

Solution used:

```
# sudo dnf install policycoreutils-python-utils -y
# sudo semanage fcontext -a -t bin_t "/linux-system-monitor/scripts/monitor.sh"
# sudo restorecon -v /linux-system-monitor/scripts/monitor.sh
# ls -Z /linux-system-monitor/scripts/monitor.sh
```

The fix preserves SELinux in Enforcing mode instead of disabling the security control. This is a practical example of diagnosing a Linux service failure using audit evidence.

12. Testing and Validation

Test 1 - Service failure detection

```
# sudo systemctl stop firewalld
# ./scripts/monitor.sh
# sudo systemctl start firewalld
# systemctl is-active firewalld
```

Expected behavior: the monitor reports firewalld as DOWN and produces a CRITICAL alert. After the service is started again, the state returns to active.

Test 2 - Network failure detection

```
Temporarily test an unreachable documentation address
# ping -c 2 -W 2 192.0.2.1
```

The script can temporarily use this test address to demonstrate the NOT CONNECTED/CRITICAL path, then be restored to the normal connectivity target.

Test 3 - CPU threshold alert

```
Temporarily lower the threshold
# CPU_THRESHOLD=1
# ./scripts/monitor.sh
```

```
Restore the normal value
# CPU_THRESHOLD=80
```

With normal CPU usage above 1%, the monitor should generate a WARNING. The threshold must be restored after testing.

Final verification commands

```
# ./scripts/monitor.sh
# tail -20 logs/monitor.log
# ls -lh reports/system-reports.txt
# systemctl status linux-monitor.timer
# systemctl list-timers --all | grep linux-monitor
# journalctl -u linux-monitor.service -n 30 --no-pager
# ls -Z scripts/monitor.sh
```


13. Screenshots

Screenshot 1: Project directory structure using tree.

```
[kunjanserver ~]$ tree /linux-system-monitor/
/linux-system-monitor/
├── config
│   └── monitor.conf
├── logs
│   └── monitor.log
├── reports
│   └── system-reports.txt
├── scripts
│   └── monitor.sh
└── systemd
    ├── linux-monitor.service
    └── linux-monitor.timer

5 directories, 6 files
[kunjanserver ~]$
```

Screenshot 2: Successful monitor.sh execution showing system information.

```
[kunjanserver ~]$ cd /linux-system-monitor/
[kunjanserver linux-system-monitor]$ ./scripts/monitor.sh

=====
      LINUX SYSTEM MONITOR
=====
Hostname       : server
OS              : Red Hat Enterprise Linux 9.8 (Plow)
Kernel         : 5.14.0-807.34.1.el9_8.x86_64
Uptime         : up 1 hour, 7 minutes

=====
      SYSTEM RESOURCES
=====
CPU Usage      : 17.50%
[OK] CPU usage is normal
Load Average   : 0.34
Memory Usage   : 39.33%
[OK] Memory usage is normal

=====
      DISK MONITORING
=====
/dev : 0%
/dev/shm : 0%
/run : 2%
/sys/firmware/efi/efivars : 23%
/ : 21%
/boot : 70%
/storage : 2%
/boot/efi : 2%
/run/user/1000 : 1%
/run/media/kunjan/CDROM : 100%
[WARNING] /run/media/kunjan/CDROM disk usage is above 80%
[WARNING] /run/media/kunjan/RHEL-9-8-BaseOS-x86_64 disk usage is above 80%

=====
      TOP PROCESSES
=====
PID COMMAND          CPU MEM
4024 gnome-shell      0.4 0.4
4190 gnome-software   1.4 3.3
4022 gnome-terminal-  0.2 0.4
398 virtiofsd         0.2 0.2
4088 systemd          0.2 0.4

=====
      NETWORK MONITORING
=====
Network Interface:
eth0      UNKNOWN 127.0.0.1/8 11/129
enp1s0    UP       192.168.233.122/24 Peer0:104717651756817568
docker0   DOWN     172.17.0.1/16

Default Route:
Default via 192.168.233.2 dev enp1s0 proto dhcp src 192.168.233.122 metric 100

Internet Connectivity:
Status : CONNECTED
[OK] Network connectivity is available

=====
      SERVICE MONITORING
=====
sshd : RUNNING
firewalld : RUNNING
[kunjanserver linux-system-monitor]$
```

Screenshot 3: CPU, load and memory monitoring with [OK] status.

```
[kunjanserver ~]$ cd /linux-system-monitor/
[kunjanserver linux-system-monitor]$ ./scripts/monitor.sh

=====
      LINUX SYSTEM MONITOR
=====
Hostname       : server
OS              : Red Hat Enterprise Linux 9.8 (Plow)
Kernel         : 5.14.0-687.34.1.el9_8.x86_64
Uptime         : up 1 hour, 11 minutes

=====
      SYSTEM RESOURCES
=====
CPU Usage      : 15.80%
[OK] CPU usage is normal
Load Average   : 0.01
Memory Usage   : 38.20%
[OK] Memory usage is normal

=====
      DISK MONITORING
=====
/dev : 0%
/dev/shm : 0%
/run : 2%
/sys/firmware/efi/efivars : 23%
/ : 21%
/boot : 70%
/storage : 2%
/boot/efi : 2%
/run/user/1000 : 1%
/run/media/kunjan/CDROM : 100%
[WARNING] /run/media/kunjan/CDROM disk usage is above 80%
/run/media/kunjan/RHEL-9-8-BaseOS-x86_64 : 100%

=====
```

Screenshot 4: Disk monitoring and top-process output.

```
Activities Terminal Sep 6 15:40
kunj@server:linux-system-monitor

[OK] Memory usage is normal
-----
DISK MONITORING
-----
/dev : 0%
/dev/shm : 0%
/run : 2%
/sys/firmware/efi/efivars : 23%
/ : 21%
/boot : 70%
/storage : 2%
/boot/efi : 2%
/run/user/1000 : 1%
/run/media/kunjan/CDROM : 100%
[WARNING] /run/media/kunjan/CDROM disk usage is above 80%
/run/media/kunjan/RHEL-9-8-0-BaseOS-x86_64 : 100%
[WARNING] /run/media/kunjan/RHEL-9-8-0-BaseOS-x86_64 disk usage is above 80%
-----
TOP PROCESSES
-----
PID COMMAND %CPU %MEM
5391 gnome-terminal- 2.6 1.4
4094 gnome-shell 1.9 5.6
4350 gnome-software 0.8 3.1
4362 vmtoolsd 0.3 1.2
998 vmtoolsd 0.2 0.2
```

Screenshot 5: Network interfaces, default route and CONNECTED status.

```
-----
NETWORK MONITORING
-----
Network Interfaces:
lo UNKNOWN 127.0.0.1/8 ::1/128
ens160 UP 192.168.211.131/24 fe80::3e6f:7f65:7bd8:375d/64
docker0 DOWN 172.17.0.1/16

Default Route:
default via 192.168.211.2 dev ens160 proto dhcp src 192.168.211.131 metric 100

Internet Connectivity:
Status : CONNECTED
[OK] Network connectivity is available
```

Screenshot 6: sshd and firewalld service status.

```
-----
SERVICE MONITORING
-----
sshd : RUNNING
firewalld : RUNNING
-----
[kunjan@server linux-system-monitor]$ s
```

Activate Windows
Go to Settings to activate Windows.

Screenshot 7: monitor.conf showing configured thresholds.

```
Activities Terminal Sep 6 15:42
kunj@server:linux-system-monitor

[kunjan@server linux-system-monitor]$ cat config/monitor.conf
# Linux System Monitor Thresholds

CPU_THRESHOLD=80
MEMORY_THRESHOLD=80
DISK_THRESHOLD=80
[kunjan@server linux-system-monitor]$
```

Screenshot 8: monitor.log containing monitoring output.

Screenshot 9: system-reports.txt containing the current report.

A screenshot of a terminal window titled "Activities" and "Terminal". The terminal shows the output of the command `cat /etc/passwd` and `cat /etc/passwd`. The output includes system information such as Hostname, OS, Kernel, Uptime, CPU Usage, Memory Usage, Disk Monitoring, Top Processes, Network Monitoring, and Service Monitoring. The terminal window has a dark background and a light-colored font. The title bar at the top shows "Sep 6 15:45" and some icons. The bottom right corner of the terminal window displays the message "Activate Windows Go to Settings to activate Windows."

Screenshot 10: linux-monitor.service configuration.

```

Activities Terminal
Sep 6 15:49
kunj@server:/linux-system-monitor
[kunj@server linux-system-monitor]$ cat systemd/linux-monitor.service
[Unit]
Description=Linux System Monitoring Tool
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/linux-system-monitor/scripts/monitor.sh
User=kunj

```

Screenshot 11: linux-monitor.timer configuration.

```
[kunj@server linux-system-monitor]$ cat systemd/linux-monitor.timer
[Unit]
Description=Run Linux System Monitor Every 5 Minutes

[Timer]
OnBootSec=1min
OnUnitActiveSec=5min
Unit=linux-monitor.service
Persistent=true

[Install]
WantedBy=timers.target
[kunj@server linux-system-monitor]$
```

Screenshot 12: systemctl status linux-monitor.timer showing active/waiting.

```

Sep 6 15:51
kunj@server:/linux-system-monitor
[kunj@server linux-system-monitor]$ systemctl status linux-monitor.timer
• linux-monitor.timer - Run Linux System Monitor Every 5 Minutes
   Loaded: loaded (/etc/systemd/system/linux-monitor.timer; enabled; preset: disabled)
   Active: active (waiting) since Sun 2026-09-06 14:26:52 IST; 1h 24min ago
     Until: Sun 2026-09-06 14:26:52 IST; 1h 24min ago
    Trigger: Sun 2026-09-06 15:52:43 IST; 1min 39s left
    Triggers: • linux-monitor.service

Sep 06 14:26:52 server systemd[1]: Started Run Linux System Monitor Every 5 Minutes.
[kunj@server linux-system-monitor]$
```

Screenshot 13: systemctl list-timers output showing the scheduled timer.

```

Sep 6 15:55
kunj@server:/linux-system-monitor — systemctl list-timers
[kunj@server linux-system-monitor]$ systemctl list-timers
NEXT                LEFT                LAST                PASSED              UNIT                                ACTIVATED
Sun 2026-09-06 15:57:43 IST 4min 22s left      Sun 2026-09-06 15:52:43 IST 37s ago             linux-monitor.timer               linux-mo
Sun 2026-09-06 17:11:38 IST 1h 18min left      Sun 2026-09-06 15:17:50 IST 35min ago             dnf-makecache.timer              dnf-make
Mon 2026-09-07 00:00:00 IST 8h left             Sun 2026-09-06 00:00:09 IST 15h ago               logrotate.timer                  logrotat
Mon 2026-09-07 00:00:00 IST 8h left             Sun 2026-09-06 00:00:09 IST 15h ago               mlocate-updatedb.timer           mlocate->
Mon 2026-09-07 14:41:36 IST 22h left            Sun 2026-09-06 14:41:36 IST 1h 11min ago           systemd-tmpfiles-clean.timer     systemd->
Sun 2026-09-13 01:00:00 IST 6 days left         Sun 2026-09-06 01:00:02 IST 14h ago               raid-check.timer                 raid-che

6 timers listed.
Pass --all to see loaded but inactive timers, too.
lines 1-10/10 (END)
```

Screenshot 14: Intentional firewalld failure showing CRITICAL alert.

```

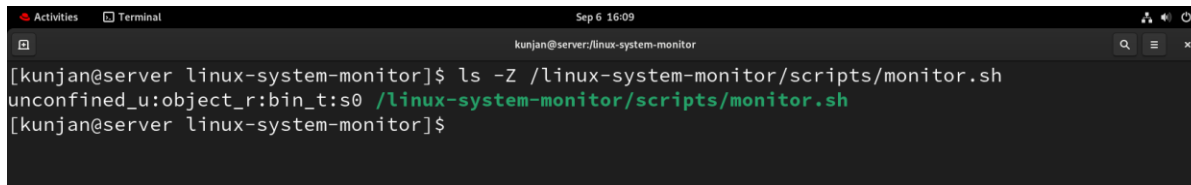
-----
SERVICE MONITORING
-----
sshd : RUNNING
firewalld : DOWN
[CRITICAL] firewalld service is not running
=====
[kunj@server linux-system-monitor]$
[kunj@server linux-system-monitor]$ systemctl stop firewalld.service

-----
SERVICE MONITORING
-----
sshd : RUNNING
firewalld : RUNNING
=====
[kunj@server linux-system-monitor]$ systemctl start firewalld.service
```

Activate Windows
Go to Settings to activate Windows.

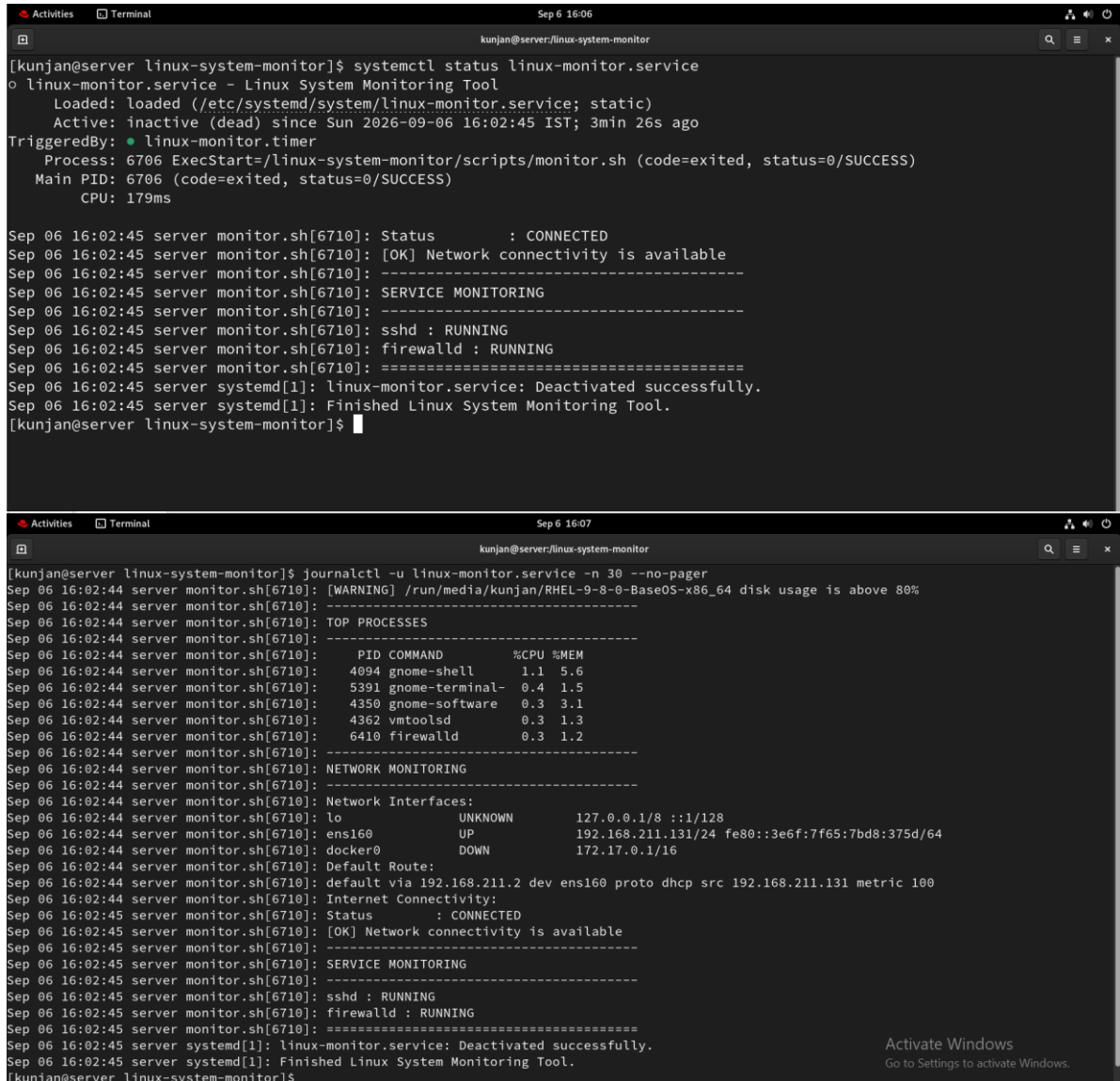
Activate Windows
Go to Settings to activate Windows.

Screenshot 15: SELinux context after the fix showing bin_t.



```
kunjan@server:linux-system-monitor
[kunjan@server linux-system-monitor]$ ls -Z /linux-system-monitor/scripts/monitor.sh
unconfined_u:object_r:bin_t:s0 /linux-system-monitor/scripts/monitor.sh
[kunjan@server linux-system-monitor]$
```

Screenshot 16: Successful systemd service execution and journal output.



```
kunjan@server:linux-system-monitor
[kunjan@server linux-system-monitor]$ systemctl status linux-monitor.service
o linux-monitor.service - Linux System Monitoring Tool
   Loaded: loaded (/etc/systemd/system/linux-monitor.service; static)
   Active: inactive (dead) since Sun 2026-09-06 16:02:45 IST; 3min 26s ago
 TriggeredBy: ● linux-monitor.timer
   Process: 6706 ExecStart=/linux-system-monitor/scripts/monitor.sh (code=exited, status=0/SUCCESS)
   Main PID: 6706 (code=exited, status=0/SUCCESS)
     CPU: 179ms

Sep 06 16:02:45 server monitor.sh[6710]: Status      : CONNECTED
Sep 06 16:02:45 server monitor.sh[6710]: [OK] Network connectivity is available
Sep 06 16:02:45 server monitor.sh[6710]: -----
Sep 06 16:02:45 server monitor.sh[6710]: SERVICE MONITORING
Sep 06 16:02:45 server monitor.sh[6710]: -----
Sep 06 16:02:45 server monitor.sh[6710]: sshd : RUNNING
Sep 06 16:02:45 server monitor.sh[6710]: firewalld : RUNNING
Sep 06 16:02:45 server monitor.sh[6710]: =====
Sep 06 16:02:45 server systemd[1]: linux-monitor.service: Deactivated successfully.
Sep 06 16:02:45 server systemd[1]: Finished Linux System Monitoring Tool.
[kunjan@server linux-system-monitor]$
```



```
kunjan@server:linux-system-monitor
[kunjan@server linux-system-monitor]$ journalctl -u linux-monitor.service -n 30 --no-pager
Sep 06 16:02:44 server monitor.sh[6710]: [WARNING] /run/media/kunjan/RHEL-9-8-0-BaseOS-x86_64 disk usage is above 80%
Sep 06 16:02:44 server monitor.sh[6710]: -----
Sep 06 16:02:44 server monitor.sh[6710]: TOP PROCESSES
Sep 06 16:02:44 server monitor.sh[6710]: -----
Sep 06 16:02:44 server monitor.sh[6710]: PID COMMAND          %CPU %MEM
Sep 06 16:02:44 server monitor.sh[6710]: 4094 gnome-shell      1.1  5.6
Sep 06 16:02:44 server monitor.sh[6710]: 5391 gnome-terminal-  0.4  1.5
Sep 06 16:02:44 server monitor.sh[6710]: 4350 gnome-software   0.3  3.1
Sep 06 16:02:44 server monitor.sh[6710]: 4362 vmttoolsd        0.3  1.3
Sep 06 16:02:44 server monitor.sh[6710]: 6410 firewalld         0.3  1.2
Sep 06 16:02:44 server monitor.sh[6710]: -----
Sep 06 16:02:44 server monitor.sh[6710]: NETWORK MONITORING
Sep 06 16:02:44 server monitor.sh[6710]: -----
Sep 06 16:02:44 server monitor.sh[6710]: Network Interfaces:
Sep 06 16:02:44 server monitor.sh[6710]: lo UNKNOWN 127.0.0.1/8 ::1/128
Sep 06 16:02:44 server monitor.sh[6710]: ens160 UP 192.168.211.131/24 fe80::3e6f:7f65:7bd8:375d/64
Sep 06 16:02:44 server monitor.sh[6710]: docker0 DOWN 172.17.0.1/16
Sep 06 16:02:44 server monitor.sh[6710]: Default Route:
Sep 06 16:02:44 server monitor.sh[6710]: default via 192.168.211.2 dev ens160 proto dhcp src 192.168.211.131 metric 100
Sep 06 16:02:44 server monitor.sh[6710]: Internet Connectivity:
Sep 06 16:02:45 server monitor.sh[6710]: Status      : CONNECTED
Sep 06 16:02:45 server monitor.sh[6710]: [OK] Network connectivity is available
Sep 06 16:02:45 server monitor.sh[6710]: -----
Sep 06 16:02:45 server monitor.sh[6710]: SERVICE MONITORING
Sep 06 16:02:45 server monitor.sh[6710]: -----
Sep 06 16:02:45 server monitor.sh[6710]: sshd : RUNNING
Sep 06 16:02:45 server monitor.sh[6710]: firewalld : RUNNING
Sep 06 16:02:45 server monitor.sh[6710]: =====
Sep 06 16:02:45 server systemd[1]: linux-monitor.service: Deactivated successfully.
Sep 06 16:02:45 server systemd[1]: Finished Linux System Monitoring Tool.
[kunjan@server linux-system-monitor]$
```

14. Security Considerations

- SELinux was kept in Enforcing mode throughout troubleshooting.
- The project directory is owned by the intended user rather than root to avoid unnecessary write-permission problems.
- The systemd service runs as the kunjan user instead of requiring the monitoring script itself to run as root.
- The monitoring tool checks firewalld status as part of basic host security monitoring.
- Thresholds are separated into a configuration file for easier administrative control.

- The project does not disable security controls simply to bypass errors; it diagnoses and fixes the underlying access policy.

15. Project Outcome and Skills Demonstrated

The completed project demonstrates practical skills in:

- RHEL system administration
- Bash scripting and shell automation
- Linux process and resource monitoring
- CPU, memory and filesystem analysis
- Linux networking and connectivity troubleshooting
- systemctl and systemd service management
- systemd timers and scheduled automation
- Linux file ownership and permissions
- SELinux context inspection and troubleshooting
- Log and report generation
- Threshold-based alerting
- Root-cause troubleshooting and verification

16. Future Enhancements

- Exclude read-only ISO and removable-media mounts from disk alerts.
- Add load-average and process-count thresholds.
- Monitor additional services through configuration rather than hard-coded checks.
- Add configurable alert destinations such as email or a messaging platform.
- Add JSON/CSV report output for integration with other tools.
- Add log rotation for long-term operation.
- Add a lightweight web dashboard if the project is expanded beyond the current Stage.

17. Conclusion

This project successfully demonstrates an end-to-end Linux system monitoring and automated alerting solution on RHEL 9.8. The tool collects system health information, evaluates configurable thresholds, monitors network connectivity and critical services, records results, generates a current report, and runs automatically through systemd.

The troubleshooting work is a significant part of the project. In particular, the systemd 203/EXEC failure was investigated through permissions, filesystem mount options and SELinux audit logs. The root cause was identified as an SELinux execution denial caused by the default_t context, and the issue was corrected without disabling SELinux. This demonstrates the practical workflow of

Monitor → Detect → Troubleshoot → Fix → Verify → Automate.

Overall, the project provides a compact but realistic demonstration of Linux administration, Bash automation, system monitoring, service management, security controls and troubleshooting.